



Universidad Nacional Autónoma de México

Facultad de Ingeniería

Ingeniería en Compputación

Sistemas Distribuidos

Reporte Proyecto 2 - Sincronización de Relojes Físicos y Lógicos

Nombre de los Integrantes:

- Ayala Hernández María Fernanda
- Saldaña Navarrete Andrea
- Tepal Briseño Hansel Yael
- Ugartechea González Luis Antonio

Profesor(a): M.I. Elba Karen Saenz García
Grupo: 3
Semestre: 2027-1
Fecha de entrega: 12 de octubre de 2026

Objetivo

Comprender algunos de los algoritmos de sincronización para manejo de relojes sicos y lógicos en un Sistema Distribuido.

Desarrollo

Realizar las siguientes actividades en equipos de 2 o tres personas. Entregar códigos fuente, así como un documento con los diagramas del ambiente sugeridos para cada actividad y capturas de pantalla que muestren el funcionamiento (se debe visualizar fecha del sistema, y en el caso de la actividad 1 el nombre de las máquinas).

Actividad 1

De acuerdo con el algoritmo de Cristian visto en clase, implementarlo usando 3 clientes, que se encuentren en máquinas diferentes, estas pueden ser sicas o virtuales y se vea una diferencia de tiempo. Agregar al código lo necesario para considerar cuando C > Cc o C<Cc. Utilizar sockets.

Solución

Utilizamos un ambiente compuesto de 4 máquinas virtuales con una distribución fedora server, por medio de Virtual Machine Manager, en donde se tiene un servidor y 3 clientes, cada uno con una diferencia de tiempo específica. El servidor se encarga de enviar la hora a los clientes, y estos a su vez ajustan su hora de acuerdo a la hora recibida del servidor. A continuación se muestra la salida para cada uno de los nodos del sistema, para el servidor:

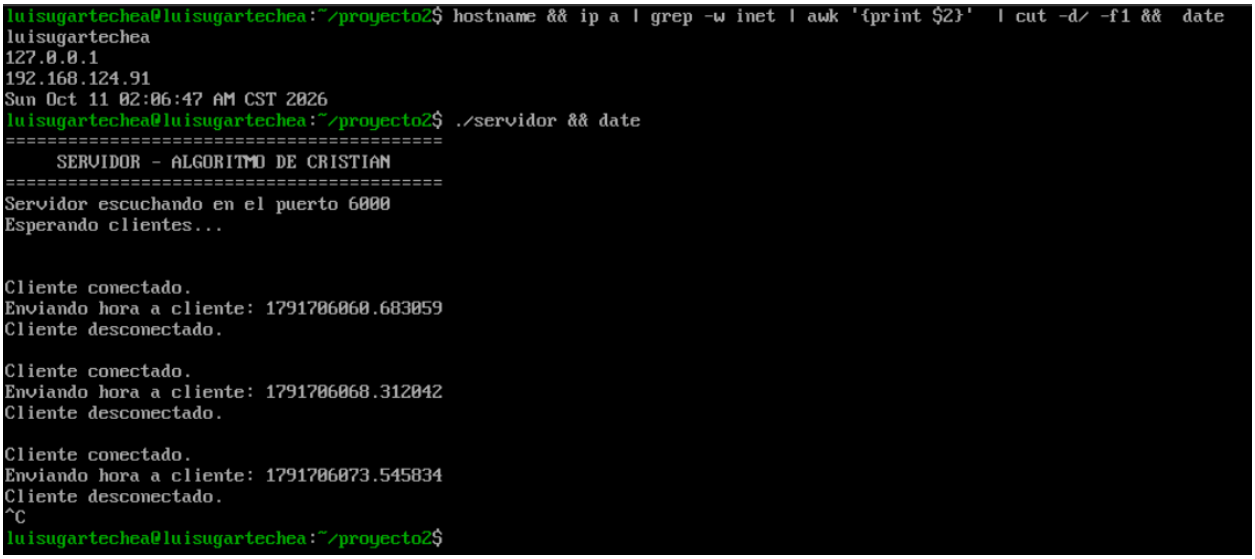


Figure 1: Ejecución del servidor para la actividad 1.

Como puede verse en la figura 1, el servidor una vez recibe la petición de los clientes, este envía el calculo de la hora a partir del desfase de tiempo que tiene cada cliente. Para asegurar que los clientes ajusten su hora de acuerdo a la hora recibida del servidor, se implementó un frenado acorde a la diferencia de tiempo que tiene cada cliente, es decir, si el cliente tiene una diferencia de tiempo de 5 segundos, este se frenará 5 segundos antes de ajustar su hora.

A continuación se muestra la salida para cada uno de los clientes, donde se puede observar que cada uno ajusta su hora de acuerdo a la hora recibida del servidor.

```
luisugartechea@luisugartechea1:~/proyecto2$ hostname && ip a | grep -w inet | awk '{print $2}' | cut -d/ -f1 && date
luisugartechea1
127.0.0.1
192.168.124.133
Sun Oct 11 02:07:01 AM CST 2026
luisugartechea@luisugartechea1:~/proyecto2$ ./cliente 192.168.124.91 15 && date
========================================
      CLIENTE - ALGORITMO DE CRISTIAN
========================================
Conectando a IP: 192.168.124.91
Desfase del reloj: +15.00 segundos

[AJUSTE] C < Cc : El reloj local se DETIENE temporalmente por 15.405612 segundos...

--------------- DATOS ----------------
T0: 1791706061.087382
Hora del servidor: 1791706060.683059
T1: 1791706061.089960
RTT: 0.002578 segundos
Retraso estimado: 0.001289 segundos
Hora ajustada: 1791706060.684348
Hora sincronizada C: 1791706076.089960
Hora sincronizada Cc: 1791706076.089960

--------------- RESULTADO ----------------
Diferencia antes:   +15.404044 segundos
Diferencia despues: +0.001289 segundos
Diferencia entre C y Cc: +0.000000 segundos
Sun Oct 11 02:07:56 AM CST 2026
luisugartechea@luisugartechea1:~/proyecto2$ _
```

Figure 2: Ejecución del cliente 1 para la actividad 1.

```
luisugartechea@luisugartechea2:~/proyecto2$ hostname && ip a | grep -w inet | awk '{print $2}' | cut -d/ -f1 && date
luisugartechea2
127.0.0.1
192.168.124.230
Sun Oct 11 02:07:15 AM CST 2026
luisugartechea@luisugartechea2:~/proyecto2$ ./cliente 192.168.124.91 -8 && date
========================================
      CLIENTE - ALGORITMO DE CRISTIAN
========================================
Conectando a IP: 192.168.124.91
Desfase del reloj: -8.00 segundos

[AJUSTE] C > Cc : El reloj de referencia se ADELANTA hasta C.

--------------- DATOS ----------------
T0: 1791706069.092571
Hora del servidor: 1791706068.312042
T1: 1791706069.094874
RTT: 0.002303 segundos
Retraso estimado: 0.001151 segundos
Hora ajustada: 1791706068.313193
Hora sincronizada C: 1791706068.313193
Hora sincronizada Cc: 1791706068.313193

--------------- RESULTADO ----------------
Diferencia antes:   -7.219807 segundos
Diferencia despues: +0.001151 segundos
Diferencia entre C y Cc: +0.000000 segundos
Sun Oct 11 02:07:49 AM CST 2026
luisugartechea@luisugartechea2:~/proyecto2$ _
```

Figure 3: Ejecución del cliente 2 para la actividad 1.

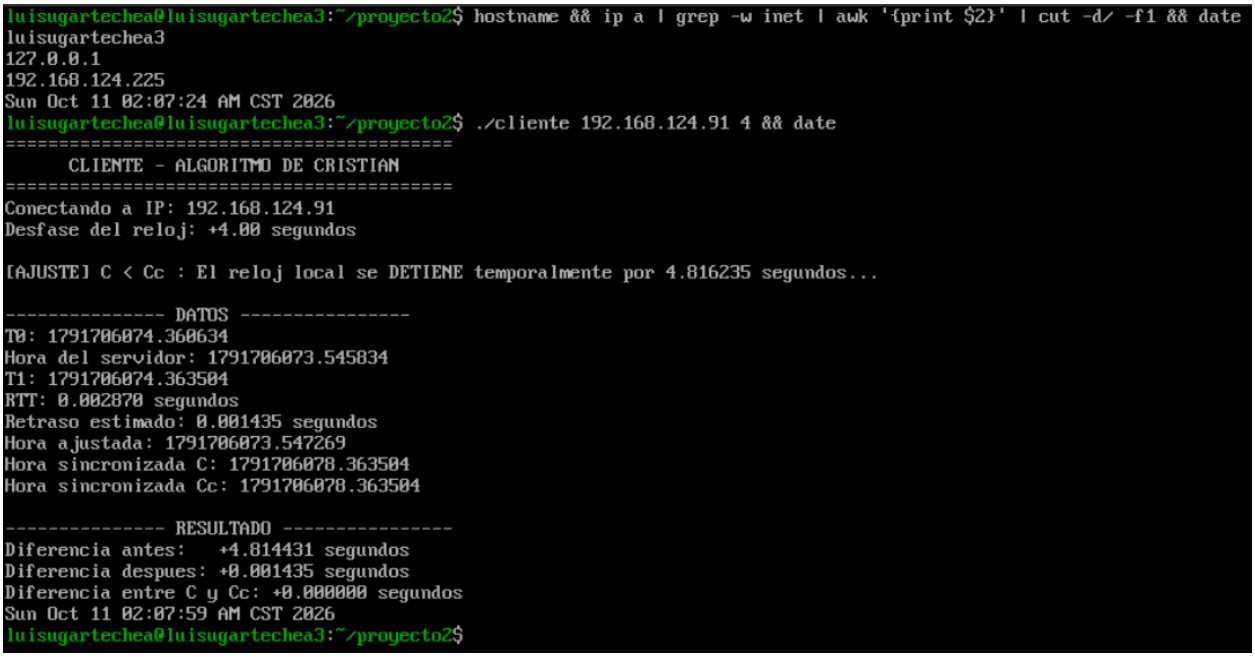


Figure 4: Ejecución del cliente 3 para la actividad 1.

Análisis de resultados

como puede observarse en las figuras 2, 3 y 4, cada uno de los clientes ajusta su hora de acuerdo a la hora recibida del servidor. El cliente uno se atrasó un total de 15 segundos, el cliente dos se adelanto un total de 8 segundos debido a que el valor que se le ingreso fue de -8 segundos y el tercer cliente se adelanto un total de 4 segundos.

Esto permite que todos los clientes tengan una hora sincronizada con el servidor, lo que es fundamental para el correcto funcionamiento de un sistema distribuido.

Actividad 2

Para la implementación del algoritmo de relojes lógicos de Lamport visto en clase, que se basa en el ambiente de eventos establecido en la figura. Se pide modificarlo utilizando un ambiente sugerido con 5 procesos y cada proceso tenga de 5 a 4 eventos. Agregar el diagrama de su ambiente.

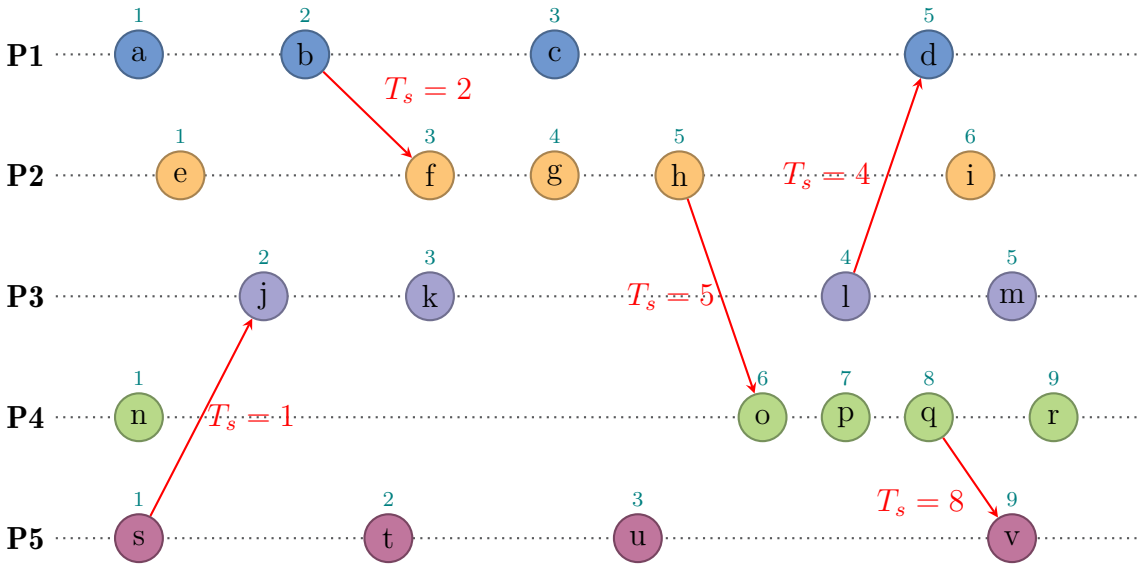


Figure 5: Ambiente sugerido para la actividad 2.

Resultados

Para validar el comportamiento del diagrama de la figura 5, se implementó el algoritmo de Lamport en un programa de python. Este programa define los 5 procesesos con sus respectivvos eventos en funciones separadas, y cada acción del algoritmo de Lamport (envio local, envio entre procesos, y recepción de mensajes) se implementa en cada función. A continuación se muestra la salida del programa donde se observa la secuencia de eventos y el calculo de los relojes lógicos para cada evento:

```
hansmarcus@pc-bda-htb:source$ python lamport_5_procesos.py
================================================
 SIMULACIÓN DE RELOJES LÓGICOS DE LAMPORT (5 PROCESOS)
================================================

[P1] Evento e1_1 en 423374 ! (LAMPORT_TIME=1)
[P2] Evento e2_1 en 423375 ! (LAMPORT_TIME=1)
[P4] Evento e4_1 en 423377 ! (LAMPORT_TIME=1)
[P1] Evento e1_2 (Mensaje enviado) desde proceso 423374 (LAMPORT_TIME=2)
[P2] Evento e2_2 (Mensaje recibido) en proceso 423375 (LAMPORT_TIME=3)
[P5] Evento e5_1 (Mensaje enviado) desde proceso 423378 (LAMPORT_TIME=1)
[P3] Evento e3_1 (Mensaje recibido) en proceso 423376 (LAMPORT_TIME=2)
[P2] Evento e2_3 en 423375 ! (LAMPORT_TIME=4)
[P5] Evento e5_2 en 423378 ! (LAMPORT_TIME=2)
[P3] Evento e3_2 en 423376 ! (LAMPORT_TIME=3)
[P2] Evento e2_4 (Mensaje enviado) desde proceso 423375 (LAMPORT_TIME=5)
[P4] Evento e4_2 (Mensaje recibido) en proceso 423377 (LAMPORT_TIME=6)
[P5] Evento e5_3 en 423378 ! (LAMPORT_TIME=3)
[P3] Evento e3_3 (Mensaje enviado) desde proceso 423376 (LAMPORT_TIME=4)
[P1] Evento e1_3 en 423374 ! (LAMPORT_TIME=3)
[P1] Evento e1_4 (Mensaje recibido) en proceso 423374 (LAMPORT_TIME=5)
[P2] Evento e2_5 en 423375 ! (LAMPORT_TIME=6)
[P4] Evento e4_3 en 423377 ! (LAMPORT_TIME=7)
[P3] Evento e3_4 en 423376 ! (LAMPORT_TIME=5)
[P4] Evento e4_4 (Mensaje enviado) desde proceso 423377 (LAMPORT_TIME=8)
[P5] Evento e5_4 (Mensaje recibido) en proceso 423378 (LAMPORT_TIME=9)
[P4] Evento e4_5 en 423377 ! (LAMPORT_TIME=9)

Simulación finalizada con éxito.
```

Figure 6: Ejecución del cliente 1 para la actividad 1.

Análisis de resultados

En la figura 6 se puede observar que los relojes lógicos de cada evento son consistentes con el diagrama de la figura 5, es decir, los eventos que ocurren antes de otros tienen un reloj lógico menor, y los eventos que ocurren después tienen un reloj lógico mayor.

Actividad 3

Con la idea anterior implementar una solución al algoritmo de relojes vectoriales, usar el ambiente planteado en la actividad anterior. Entregar la figura con el ambiente sugerido que se programará y con el desarrollo del algoritmo, es decir el reloj vectorial de los eventos.

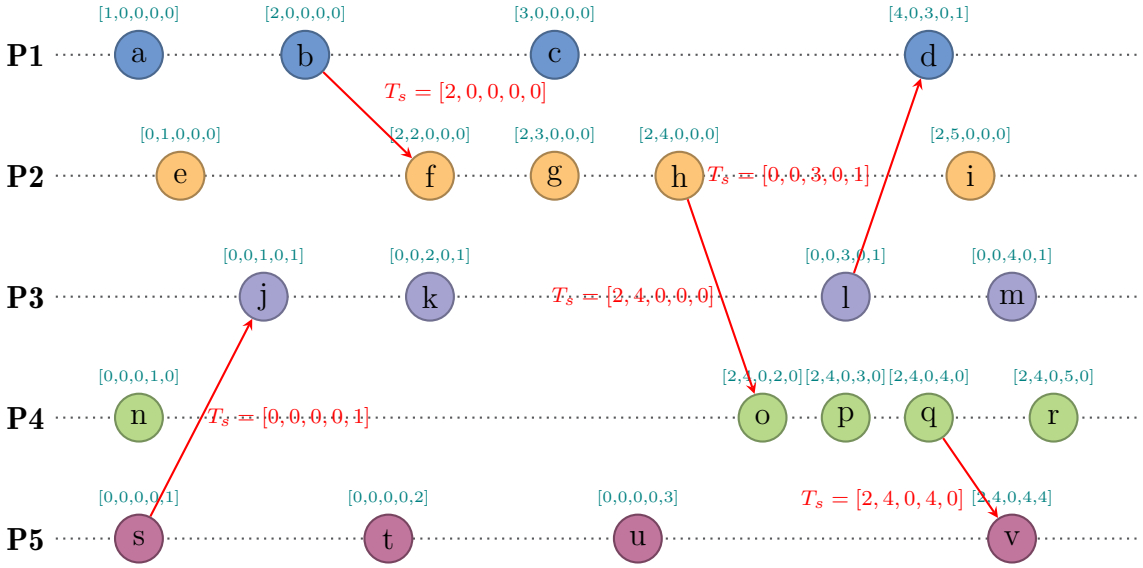


Figure 7: Ambiente sugerido para la actividad 3 con Relojes Vectoriales.

Resultados

Con la implementación del algoritmo de relojes vectoriales, nos permitio a diferencia del algoritmo de Lamport, conocer que eventos de nuestro sistema distribuido eran concurrentes, es decir, que no existe relación alguna que provoque que un evento ocurra antes que otro, y por lo tanto no se puede determinar un orden entre ellos.

Para su implementacion se partío del mismo programa de python que se utilizó para la actividad 2, y se modificó para que cada proceso mantenga un vector de relojes, y cada vez que ocurre un evento, se actualiza el vector de relojes correspondiente. A continuación se muestra la salida del programa donde se observa la secuencia de eventos y el calculo de los relojes vectoriales para cada evento:

```
hansmarcus@pc-bda-htb:source$ python vectorial_5_procesos.py
================================================
 SIMULACIÓN DE RELOJES VECTORIALES (5 PROCESOS)
================================================

[P1] Evento e1_1 en 426518 ! (VECTOR_TIME=[1, 0, 0, 0, 0])
[P2] Evento e2_1 en 426519 ! (VECTOR_TIME=[0, 1, 0, 0, 0])
[P4] Evento e4_1 en 426521 ! (VECTOR_TIME=[0, 0, 0, 1, 0])
[P1] Evento e1_2 (Mensaje enviado) desde proceso 426518 (VECTOR_TIME=[2, 0, 0, 0, 0])
[P2] Evento e2_2 (Mensaje recibido) en proceso 426519 (VECTOR_TIME=[2, 2, 0, 0, 0])
[P5] Evento e5_1 (Mensaje enviado) desde proceso 426522 (VECTOR_TIME=[0, 0, 0, 0, 1])
[P3] Evento e3_1 (Mensaje recibido) en proceso 426520 (VECTOR_TIME=[0, 0, 1, 0, 1])
[P2] Evento e2_3 en 426519 ! (VECTOR_TIME=[2, 3, 0, 0, 0])
[P5] Evento e5_2 en 426522 ! (VECTOR_TIME=[0, 0, 0, 0, 2])
[P3] Evento e3_2 en 426520 ! (VECTOR_TIME=[0, 0, 2, 0, 1])
[P2] Evento e2_4 (Mensaje enviado) desde proceso 426519 (VECTOR_TIME=[2, 4, 0, 0, 0])
[P4] Evento e4_2 (Mensaje recibido) en proceso 426521 (VECTOR_TIME=[2, 4, 0, 2, 0])
[P5] Evento e5_3 en 426522 ! (VECTOR_TIME=[0, 0, 0, 0, 3])
[P3] Evento e3_3 (Mensaje enviado) desde proceso 426520 (VECTOR_TIME=[0, 0, 3, 0, 1])
[P1] Evento e1_3 en 426518 ! (VECTOR_TIME=[3, 0, 0, 0, 0])
[P1] Evento e1_4 (Mensaje recibido) en proceso 426518 (VECTOR_TIME=[4, 0, 3, 0, 1])
[P2] Evento e2_5 en 426519 ! (VECTOR_TIME=[2, 5, 0, 0, 0])
[P4] Evento e4_3 en 426521 ! (VECTOR_TIME=[2, 4, 0, 3, 0])
[P3] Evento e3_4 en 426520 ! (VECTOR_TIME=[0, 0, 4, 0, 1])
[P4] Evento e4_4 (Mensaje enviado) desde proceso 426521 (VECTOR_TIME=[2, 4, 0, 4, 0])
[P5] Evento e5_4 (Mensaje recibido) en proceso 426522 (VECTOR_TIME=[2, 4, 0, 4, 4])
[P4] Evento e4_5 en 426521 ! (VECTOR_TIME=[2, 4, 0, 5, 0])

Simulación finalizada con éxito.
hansmarcus@pc-bda-htb:source$
```

Figure 8: Ejecución del cliente 1 para la actividad 1.

Análisis de resultados

Con esta salida podemos comprobar que los eventos **a**, **b**, **n** y **s** son concurrentes ya que cumplen la condición de que los vectores son bidireccionales. En el diagrama puede verse la concurrecnia en otros eventos como **c** hacia cualquier evento de los procesos 2, 4 y 5.

Conclusiones

Para finalizar, podemos concluir que los algoritmos de sincronización de relojes son fundamentales para el correcto funcionamiento de un sistema distribuido, ya que permiten mantener un orden lógico de los eventos y garantizar la consistencia de los datos impidiendo

problemas de sincronización. La implementación de estos algoritmos nos permitió comprender mejor su funcionamiento y aplicarlos en situaciones prácticas. Además de repasar los conceptos teóricos vistos en clase, así como la importancia de elegir el algoritmo adecuado según las necesidades del sistema.